LangChain

Advanced Study Guide

RAG Systems · Tools · Tool Calling · AI Agents

Videos 15–18 • Nitesh's Hindi YouTube Tutorial Series

V15 RAG Project	V16 Tools	V17 Tool Calling	V18 AI Agents

Table of Contents

Section 1 — YouTube RAG Chat System (Video 15)

YouTube RAG Chat System

Build a system that lets you chat with any YouTube video in real time. Ask questions, get summaries, or resolve doubts — all powered by a RAG pipeline built with LangChain.

Problem Statement

- Long YouTube videos (podcasts, lectures) are hard to digest
- Users want to ask questions about a video without watching all of it
- Solution: RAG-based system that indexes the transcript and answers queries

RAG Architecture — 4 Steps

① **Indexing — Load & Chunk**
Fetch transcript via YouTube Transcript API → split with RecursiveCharacterTextSplitter (chunk_size=1000, overlap=200) → embed with OpenAI Embeddings → store in FAISS vector store

② **Retrieval**
Build a similarity-search retriever from the vector store → invoke with user query → returns top-4 relevant document chunks

③ **Augmentation**
Merge retrieved chunks into one context string → inject into PromptTemplate with {context} and {question} variables

④ **Generation**
Send final prompt to LLM → parse response → return answer to user

Step-by-Step Code Walkthrough

Step 1 — Load Transcript

- Use `youtube_transcript_api` — more reliable than LangChain's YouTube loader
- Pass video ID (not full URL) + language code (e.g., 'en' or 'hi')
- Returns list of dicts (text + timestamp) → join all into one big string

```
from youtube_transcript_api import YouTubeTranscriptApi
transcript_list = YouTubeTranscriptApi.get_transcript(video_id, languages=['en'])
transcript = " ".join([t["text"] for t in transcript_list])
```

Step 2 — Split into Chunks

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.create_documents([transcript])  # ~168 chunks for 2hr video
```

Step 3 — Embed & Store in Vector DB

```
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
```

Step 4 — Build Retriever & Chain

```
retriever = vectorstore.as_retriever(search_type='similarity', search_kwargs={'k': 4})
from langchain_core.prompts import PromptTemplate
prompt = PromptTemplate(
    template="You are a helpful assistant. Answer only from the provided transcript contex
t.
    If context is insufficient, say you don't know.\n
    Context: {context}\nQuestion: {question}",
    input_variables=["context", "question"]
)
# Final chain using RunnableParallel + RunnableLambda
from langchain.schema.runnable import RunnableParallel, RunnablePassthrough, RunnableLambd
a
def format_docs(docs):
    return "\n".join([d.page_content for d in docs])
parallel_chain = RunnableParallel({
    'context': retriever | RunnableLambda(format_docs),
    'question': RunnablePassthrough()
})
final_chain = parallel_chain | prompt | llm | parser
result = final_chain.invoke("What is DeepMind?")
```

Key RAG Chain Insight

• The chain uses TWO parallel branches: one for context (retriever → format_docs) and one for question (passthrough)

• RunnableLambda wraps format_docs so it can participate in the chain as a Runnable

• Everything connects with the pipe | operator — one invoke() triggers the entire pipeline

• RunnablePassthrough() simply passes the input question unchanged to the prompt

Tools in LangChain

LLMs can think and speak, but they cannot act. Tools give LLMs hands and legs — the ability to perform real-world tasks like searching the web, booking tickets, or querying a database.

Why LLMs Need Tools

LLM Can Do	LLM Cannot Do
Understand & reason about a question	Fetch live weather data from an API
Generate text & language output	Reliably perform complex math
Break down a problem into steps	Book a train ticket on IRCTC
Explain concepts & summarize content	Post a tweet on your behalf
Answer from parametric knowledge	Query your own database

Analogy: An LLM is like a human body with a brain (can think & speak) but no hands or legs (cannot take action). Tools provide those hands and legs.

What is a Tool?

A Tool is a Python function packaged so the LLM can understand and call it when needed. Every tool has 3 key attributes:

Attribute	Description	Example
`.name`	The tool's identifier	"multiply"
`.description`	Tells the LLM what this tool does (from docstring)	"Multiplies two numbers A and B"
`.args`	Input schema — what format the tool expects	{"a": int, "b": int}

Two Types of Tools

2.1 Built-in Tools

LangChain pre-builds tools for common tasks. Just import and use — no custom code needed.

Tool	Import	Use Case
DuckDuckGoSearchRun	langchain_community.tools	Real-time web search
WikipediaQueryRun	langchain_community.tools	Wikipedia article summaries
PythonREPLTool	langchain_community.tools	Execute raw Python code

ShellTool	langchain_community.tools	Run shell/CLI commands
RequestsGetTool	langchain_community.tools	Make HTTP GET requests
GmailSendMessage	langchain_community.tools	Send emails via Gmail
SQLDatabaseQueryTool	langchain_community.tools	Run SQL queries

■ *ShellTool is powerful but risky in production — it can delete files. Use carefully.*

2.2 Custom Tools — 3-Step Process

When no built-in tool exists for your use case, build your own:

1 **Write the function**
Plain Python function with your business logic

2 **Add type hints + docstring**
Type hints tell LLM the input/output types. Docstring becomes the tool description

3 **Apply @tool decorator**
Transforms the function into a LangChain Tool (Runnable) that the LLM can interact with

```
from langchain_core.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers A and B and return their product."""
    return a * b

# Tools are Runnables — use invoke() just like any other component
result = multiply.invoke({"a": 3, "b": 5})  # → 15

# Inspect the tool
print(multiply.name)         # 'multiply'
print(multiply.description)  # 'Multiply two numbers A and B...'
print(multiply.args)         # {'a': int, 'b': int}
```

When to Build Custom Tools

- You need to call your own internal/company APIs
- You want to encapsulate custom business logic
- Your agent needs to interact with your existing database or product
- No existing built-in tool covers your specific use case

Agent Definition (Preview)

An AI Agent is an LLM-powered system that can autonomously think, decide, and take actions using external tools and APIs to achieve a goal. In short: Agent = LLM (reasoning) + Tools (action).

Tool Calling & Binding

Tool calling is the complete workflow that connects an LLM to a tool. It involves 4 distinct steps. Understanding each step is critical for building AI agents.

The Complete 4-Step Tool Calling Flow

① **Tool Creation**
Build a Python function decorated with `@tool` (covered in Video 16)

② **Tool Binding**
Register the tool with an LLM using bind_tools(). After binding, the LLM knows:
- What tools are available
- What each tool does (from description)
- What input format each tool expects (from type hints)

```
llm_with_tools = llm.bind_tools([multiply])
# Store in a new variable — the original LLM is unchanged
```

③ **Tool Calling**
When a relevant query is sent, the LLM SUGGESTS (does not execute) which tool to call and with what inputs. The LLM returns a tool_calls list instead of normal text content.

```
result = llm_with_tools.invoke('What is 8 * 7?')
# result.content → '' (empty!)
# result.tool_calls → [{'name': 'multiply', 'args': {'a': 8, 'b': 7}, 'id': '..
.'}]
```

④ **Tool Execution**
You (the programmer) execute the tool using the LLM's suggested name and args. Result is returned as a ToolMessage which can be sent back to the LLM for final response.

```
tool_call = result.tool_calls[0]
tool_result = multiply.invoke(tool_call)  # Pass entire tool_call object
# Returns a ToolMessage (not just the raw result)
```

Critical Insight: LLMs Don't Execute Tools

A major misconception: the LLM does NOT call the tool. It only suggests which tool to call and with what arguments. The actual execution is done by LangChain / your code. This is intentional — it keeps control with the programmer and avoids risky autonomous actions.

Full End-to-End Flow with Message History

To get the LLM's final natural-language response after tool execution, you must maintain a message history and send it back to the LLM:

```
from langchain_core.messages import HumanMessage
query = 'Can you multiply 3 with 1000?'
messages = [HumanMessage(query)]
# Step 1: First LLM call → triggers tool calling
ai_msg = llm_with_tools.invoke(messages)
messages.append(ai_msg)  # Add AI message (with tool_calls) to history
# Step 2: Execute the tool
tool_result = multiply.invoke(ai_msg.tool_calls[0])  # Returns ToolMessage
messages.append(tool_result)  # Add tool result to history
# Step 3: Send full history → LLM generates final response
final_response = llm_with_tools.invoke(messages)
print(final_response.content)  # 'The product of 3 and 1000 is 3000'
```

Message Types in Tool Calling

Message Type	When It Appears	Content
HumanMessage	User's question	Natural language query
AIMessage	LLM's tool call suggestion	Empty content + tool_calls list with name & args
ToolMessage	After tool execution	Tool's return value; linked to AIMessage by tool_call_id
AIMessage (final)	LLM's natural response	Human-readable answer based on tool result

Real-World Example: Currency Converter

Demo app: A tool calls the ExchangeRate API to fetch live conversion rates, then the LLM uses it to answer real-time currency questions.

```
@tool
def get_conversion_factor(base_currency: str, target_currency: str) -> float:
    """Get the real-time conversion factor between two currencies."""
    response = requests.get(f'https://api.exchangerate-api.com/v4/latest/{base_currency}')
    data = response.json()
    return data['rates'][target_currency]
# LLM can now answer: 'What is 100 USD in INR?' with real-time data
```

AI Agents

An AI Agent is an intelligent system that receives a high-level goal and autonomously plans, decides, and executes a sequence of actions using external tools, APIs, and knowledge sources — while maintaining context and adapting to new information.

Motivating Use Case: Trip Planning

Planning a Delhi → Goa trip involves: searching flights & trains, comparing prices, booking hotels, building an itinerary, cab bookings, and calendar setup. This takes days manually. An AI agent does it in minutes via a single chat interface.

LLM vs AI Agent

	Plain LLM	AI Agent
Reasoning	✓ Yes	✓ Yes (via LLM)
Output generation	✓ Yes	✓ Yes (via LLM)
Take real-world action	✗ No	✓ Yes (via Tools)
Multi-step planning	✗ Limited	✓ Autonomous
Maintain context	✗ Single call	✓ Across many steps
Adapt to new info	✗ No	✓ Mid-task replanning

Formula: AI Agent = LLM (reasoning engine) + Tools (action capabilities)

5 Core Characteristics of AI Agents

Goal-driven	Give the agent a high-level objective — it figures out HOW to achieve it
Plans autonomously	Breaks down complex goals into sequential steps and executes them
Tool-aware	Knows which tools it has and when/how to use each one
Context-maintaining	Keeps full memory of what it has done and what the user prefers

Adaptive	Replans dynamically if something goes wrong (e.g., API failure → use backup)

ReAct Design Pattern

ReAct (Reasoning + Action) is the most popular agent design pattern. The agent alternates between reasoning about what to do next and taking an action (using a tool), then observing the result and reasoning again — until the goal is achieved.

Phase	What Happens
Reason	LLM thinks: 'To answer this, I need to search the web'
Act	AgentExecutor calls the search tool with a query
Observe	Tool returns results; LLM reads the observation
Reason again	LLM decides: 'I have enough info to answer now'
Final answer	LLM generates human-readable response

Agent vs AgentExecutor

Component	Role	Analogy
Agent	Plans, reasons, decides which tool to use and when	The brain — does all the thinking
AgentExecutor	Actually executes the tools the Agent decides to use	The hands — does the doing

Building a ReAct Agent in LangChain

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI

# 1. Set up tools
search_tool = DuckDuckGoSearchRun()
tools = [search_tool]

# 2. Set up LLM
llm = ChatOpenAI(model='gpt-4o')

# 3. Pull pre-built ReAct prompt from LangChain Hub
prompt = hub.pull('hwchase17/react')

# 4. Create the ReAct agent
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
```

```
# 5. Wrap in AgentExecutor (handles the reasoning loop)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 6. Run the agent
result = agent_executor.invoke({'input': 'What is the top news in India today?'})
print(result['output'])
```

Note: Why provide tools twice (to create_react_agent AND AgentExecutor)? create_react_agent tells the LLM which tools exist. AgentExecutor needs them to actually execute them. They serve different purposes.

Quick Reference

RAG Pipeline Cheat Sheet

Step	Component	Code
Load	YouTubeTranscriptApi	`YouTubeTranscriptApi.get_transcript(id, languages=['en'])`
Split	RecursiveCharacterTextSplitter	`splitter.create_documents([text])`
Embed+Store	FAISS + OpenAIEmbeddings	`FAISS.from_documents(chunks, embeddings)`
Retrieve	FAISS retriever	`vectorstore.as_retriever(search_kwargs={'k': 4})`
Augment	PromptTemplate	`prompt.invoke({'context': ..., 'question': ...})`
Generate	LLM + Parser	`llm.invoke(prompt) → parser.invoke(result)`

Custom Tool Creation Steps

Step	What to Do	Why
1	Write the Python function with logic	Define what the tool does
2	Add type hints to parameters and return	LLM knows input/output format
3	Add docstring describing the tool	LLM knows when to use this tool
4	Apply @tool decorator above the function	Makes it a LangChain Runnable

Tool Calling Flow Summary

Step	Action	Message Type
1	User sends query	HumanMessage
2	LLM detects tool need → generates tool_calls	AIMessage (tool_calls not empty)
3	Programmer executes tool with suggested args	—
4	Tool returns result wrapped in ToolMessage	ToolMessage

5	Full message history sent back to LLM	HumanMessage + AIMessage + ToolMessage
6	LLM generates final natural-language response	AIMessage (final)

Import Reference — Videos 15–18

Component	Import Path
YouTubeTranscriptApi	youtube_transcript_api
RecursiveCharacterTextSplitter	langchain.text_splitter
FAISS	langchain.vectorstores
OpenAIEmbeddings	langchain_openai
tool decorator	langchain_core.tools
DuckDuckGoSearchRun	langchain_community.tools
ShellTool	langchain_community.tools
HumanMessage	langchain_core.messages
create_react_agent	langchain.agents
AgentExecutor	langchain.agents
hub (for prompts)	langchain
RunnableParallel	langchain.schema.runnable
RunnablePassthrough	langchain.schema.runnable
RunnableLambda	langchain.schema.runnable